ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÀI TẬP LỚN 1: TÌM KIẾM

Pipes và Light Up

Nhập môn Trí tuệ nhân tạo (CO3061)
Học kỳ 1, năm học 2026–2027

Giảng viên: Ths. Vương Bá Thịnh
Lớp: L01

Hà Bảo Nhi	2312496
Lê Chí Đại	2310621
Lương Minh Thuận	2313348
Nguyễn Quốc Huy	2311209
Đào Quang Dương	2310579

Nội dung

1. Pipes

1.1 Giới thiệu
1.2 Phát biểu bài toán tìm kiếm
1.3 Tạo dữ liệu đầu vào
1.4 Giải thuật tìm kiếm mù
1.5 Giải thuật tìm kiếm heuristic
1.6 Hiện thực và minh họa từng bước
1.7 Kết quả thực nghiệm
1.8 Nhận xét

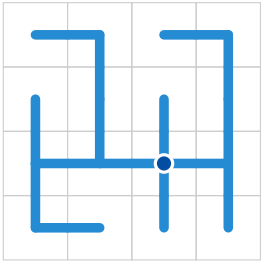
2. Light Up (Akari)

2.1 Giới thiệu
2.2 Phát biểu bài toán tìm kiếm
2.3 Tạo dữ liệu đầu vào
2.4 Giải thuật tìm kiếm mù
2.5 Giải thuật tìm kiếm heuristic
2.6 Hiện thực và minh họa từng bước
2.7 Kết quả thực nghiệm
2.8 Nhận xét

Mỗi game: một giải thuật mù, một giải thuật heuristic, demo từng bước và bảng đo thời gian/bộ nhớ.

1.1 Pipes

Giới thiệu



Một nghiệm của pipes-4x4-261.

- Xoay từng ống; hình dạng ống giữ nguyên.
- Các cổng phải khớp nhau, không mở ra ngoài biên.
- Mọi ô nối thành một mạng, không có chu trình.

Hai giải thuật: DFS và Greedy Best-First.

1.2 Pipes

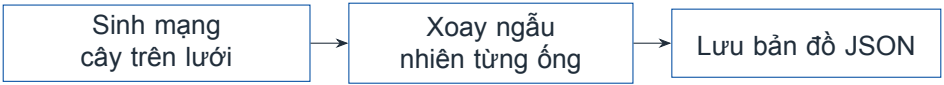
Phát biểu bài toán tìm kiếm

Thành phần	Biểu diễn
Trạng thái	$s = (a_0, \ldots, a_{k-1})$: hướng đã gán cho k ô đầu
Ban đầu	$()$, chưa gán hướng cho ô nào
Hành động	Gán một hướng cho ô k, theo thứ tự hàng–cột
Đích	Gán đủ $n = m^2$ ô và mạng thỏa luật
Không gian	$\prod_i \lvert R_i \rvert \leq 4^n$ cấu hình trước kiểm tra ràng buộc

Ống thẳng có 2 hướng; chữ thập có 1; các hình còn lại có 4.
Giải thuật gán thẳng hướng cuối cho một ô (bằng 0–3 lần nhấp xoay).

1.3 Pipes

Tạo dữ liệu đầu vào



- 9 bản: 4×4, 5×5, 7×7; mỗi cỡ 3 seed.
- JSON lưu kích thước, nguồn và ma trận cổng N/E/S/W.
- Mạng cây bảo đảm có nghiệm; solver không đọc đáp án lúc sinh.

Luật và dạng ống lấy theo puzzle-pipes.com; bàn do chương trình tự sinh.

Giải thuật tìm kiếm mù: DFS

```
DFS(s):
  if limit reached: return LIMIT
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):
      result = DFS(child)
      if solved or limited:
        return result
  return UNSOLVABLE
```

Kiểm tra ở gốc: tổng số cổng phải bằng $2(n-1)$ vì mạng là một cây.

Đi sâu theo thứ tự cố định; bế tắc thì quay lui.

Cắt tỉa dùng chung:

- Cổng vượt biên.
- Không khớp ô đã gán.
- Đã tạo chu trình.
- Không thể nối hết ô.

Giải thuật tìm kiếm heuristic: Greedy Best-First

$$h(s) = \sum_{i \text{ chưa gán}} \log_2 |D_i(s)|$$

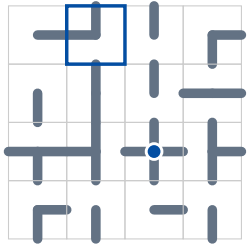
$D_i(s)$ là miền hướng còn lại của ô i. Ưu tiên h nhỏ hơn.
Ví dụ: miền $(1, 2, 4)$ cho $h = 3$; miền $(1, 1, 2)$ cho $h = 1$.

```
push root into min-heap
while heap not empty:
  s = pop state with lowest priority
  if GOAL(s): return SOLVED(s)
  for child in feasible successors(s):
    push (h(child), -depth(child), serial, child)
return UNSOLVABLE
```

Cùng thứ tự ô và cắt tỉa với DFS; chỉ khác thứ tự lấy trạng thái ra. Hòa h thì chọn trạng thái sâu hơn.

1.6 Pipes

Hiện thực và minh họa từng bước



Ví dụ bước gán ô thứ hai:

1. Ô đầu có cổng Đông.
2. Thử ES ở ô kế: thiếu cổng Tây ⇒ loại.
3. Thử SW: khớp ô đầu ⇒ đi tiếp.

Trên game: chọn DFS/Greedy, dùng Run, Pause, Next; xem lý do loại nhánh và Show solution.

Python 3, pygame-ce. Luật ở model.py; giải thuật ở search.py; giao diện ở app.py.

1.7 Pipes

Kết quả thực nghiệm

Cỡ	Giải thuật	Giải được/lượt	Thời gian (ms)	Peak (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

9 bản $\times$ 2 giải thuật $\times$ 5 lần lặp = 90 lượt, đều giải được.
Giới hạn mỗi lượt: 30 giây và 200.000 lần mở rộng.

Trung vị theo cỡ bàn. Thời gian đo không vẽ giao diện; bộ nhớ là đỉnh cấp phát Python (tracemalloc). Bảng từng bàn trong báo cáo.

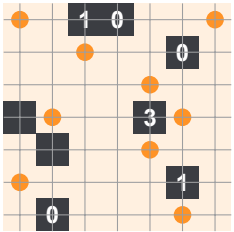
Nhận xét

- Hai giải thuật nhanh gần bằng nhau; DFS nhỉnh hơn ở 5×5 và 7×7.
- Lý do: cắt tỉa đã loại gần hết hướng sai, DFS gần như đi thẳng tới nghiệm. Greedy còn tốn thời gian tính *h* và thao tác heap.
- Greedy dùng ít bộ nhớ hơn 2–5 lần: heap rất nhỏ, còn DFS giữ bản sao trạng thái ở mỗi mức.

Hạn chế: mới có 9 bàn khá dễ, chưa có bàn vô nghiệm trong bảng đo.

2.1 Light Up (Akari)

Giới thiệu



- Đặt bóng trên ô trắng, chiếu theo hàng/cột đến tường.
- Mọi ô trắng phải sáng.
- Hai bóng không nhìn thấy nhau.
- Tường số có đúng số bóng kề trực tiếp.

Hai giải thuật: DFS và CSP (backtracking + MRV + CP).

Luật theo Nikoli. Hình: bàn 7×7 seed 70097 và lời giải CSP tìm được.

2.2 Light Up (Akari)

Phát biểu bài toán tìm kiếm

Thành phần	Biểu diễn
Trạng thái	Tập bóng, ô sáng và vị trí còn hợp lệ (bitmask)
Ban đầu	Chưa có bóng; CSP có thể suy ra bóng ngay tại gốc
Hành động	Đặt bóng; CSP có thể chọn một tổ hợp quanh tường
Đích	Sáng hết ô trắng, không xung đột, mọi số trên tường đúng
Không gian	2^W tập bóng trước ràng buộc, với W ô trắng

Một nút CSP có thể đặt nhiều bóng cùng lúc nhờ suy luận.
Cả hai tìm lời giải đầu tiên, không tối ưu số bóng.

2.3 Light Up (Akari)

Tạo dữ liệu đầu vào

1. Dùng seed chọn vị trí tường.
2. Đặt bóng làm chứng đến khi mọi ô trắng sáng.
3. Suy ra số trên tường; giữ một phần tường không số.
4. Kiểm chứng lời giải làm chứng trước khi trả bàn.

4 cỡ: $7, 10, 14, 25$; 3 nhãn; 5 seed khác nhau mỗi cấu hình.
Tổng cộng **60 bản**, mỗi bản dùng chung cho hai giải thuật.

easy/normal/hard khác nhau ở tỉ lệ tường: 16%, 10.5%, 5.5%. Solver không biết lời giải mẫu.

2.4 Light Up (Akari)

Giải thuật tìm kiếm mù: DFS

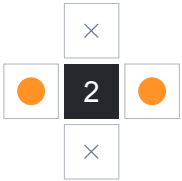
```
DFS(state):
  if limit reached: return LIMIT
  if GOAL(state): return SOLVED
  target = first dark cell in row-major order
  for bulb in legal illuminators(target):
    child = place(bulb)
    if FEASIBLE(child):
      result = DFS(child)
      if solved or limited: return result
    exclude bulb from later sibling choices
  return UNSOLVABLE
```

Đi sâu theo thứ tự cố định. Loại nhánh nếu tường không đủ chỗ hoặc ô tối không còn nguồn chiếu.

Trong code, đệ quy được thay bằng stack tường minh.

2.5 Light Up (Akari)

Giải thuật tìm kiếm heuristic: MRV và CP



Tường đủ 2 bóng: cấm hai ô kề còn lại.

CP: lan truyền quyết định bắt buộc

- Tường thiếu *r* bóng, còn đúng *r* ô: đặt tất cả.
- Ô tối còn một nguồn chiếu: đặt bóng tại nguồn đó.
- Bóng mới cấm các vị trí nhìn thấy nhau.

Lặp đến khi không đổi; mâu thuẫn thì quay lui.

Giải thuật tìm kiếm heuristic: chọn nhánh bằng MRV

Chọn ràng buộc ít lựa chọn nhất: đếm nguồn chiếu của ô tối, hoặc $\binom{u}{r}$ tổ hợp quanh tường còn u ô và thiếu r bóng.

```
BACKTRACK(state):
  check limits; propagate until no change
  if contradiction: return failure
  if GOAL(state): return SOLVED
  select constraint by MRV
  for choice ordered by lighting gain:
    result = BACKTRACK(apply(choice, checkpoint))
    if solved or limited: return result
    restore checkpoint
  return failure
```

Bằng điểm thì ưu tiên tường. Lựa chọn chiếu sáng thêm nhiều ô được thử trước.

2.6 Light Up (Akari)

Hiện thực và minh họa từng bước

1. Chọn DFS hoặc CSP, bấm Start Search.
2. Pause và hai mũi tên để đi tới/lui từng bước.
3. CSP hiện thêm bóng suy ra, ô bị cấm $\times$ và lúc quay lui.
4. Compare DFS and CSP: chạy cả hai trên bàn đang mở.

Solver chạy trong worker riêng; giao diện nhận từng sự kiện để vẽ.

Python 3, pygame-ce. Luật ở core/, giải thuật ở algorithms/search.py, giao diện ở ui/app.py.

2.7 Light Up (Akari)

Kết quả thực nghiệm: số bản giải được

	DFS		CSP	
Cỡ	Giải được	TIMEOUT	Giải được	TIMEOUT
7×7	15/15	0	15/15	0
10×10	15/15	0	15/15	0
14×14	15/15	0	15/15	0
25×25	7/15	8	15/15	0
Tổng	52/60	8	60/60	0

60 bản $\times$ 2 giải thuật = 120 lượt. Mỗi bản đo một lần mỗi giải thuật.
Giới hạn: 30 giây, 500.000 node, 512 MiB ước lượng.

Mỗi lượt chạy trong process mới. Không lượt nào chạm giới hạn nút hoặc bộ nhớ.

2.7 Light Up (Akari)

Kết quả thực nghiệm: thời gian và bộ nhớ

Input (cỡ/nhãn)	Giải thuật	Giải được	Giây	KiB
14 / easy	DFS	5/5	0.2330	135.3
	CSP	5/5	0.0801	109.1
25 / easy	DFS	2/5	30.0001	2237.0
	CSP	5/5	0.9558	562.1
25 / normal	DFS	1/5	30.0004	3583.3
	CSP	5/5	1.0274	696.9
25 / hard	DFS	4/5	0.7835	870.3
	CSP	5/5	1.0421	868.0

Trung vị 5 seed, tính cả lượt TIMEOUT (30 giây là lúc bị dừng). Bàn 7 và 10: hai giải thuật đều dưới 0.04 giây. Bảng đầy đủ trong báo cáo.

Nhận xét

- CSP giải đủ 60/60 bàn, DFS 52/60; 8 lượt TIMEOUT đều ở bàn 25×25.
- Bàn lớn: DFS mở hàng nghìn nút, CSP chỉ khoảng 30 nút nhờ suy ra bóng bắt buộc.
- Bàn nhỏ: DFS lại nhanh hơn (37/52 bàn) vì CSP tốn công suy luận ở mỗi nút.

Hạn chế: mỗi cấu hình mới 5 seed; nhãn độ khó chỉ dựa trên số tường; chưa đo riêng tác dụng MRV và CP.

Kết luận

- Hai game viết bằng Python 3, có chơi tay và xem giải thuật chạy từng bước.
- Pipes: DFS và Greedy giải hết 9 bàn, tốc độ gần như nhau.
- Light Up: CSP giải 60/60, DFS 52/60; CSP vượt trội ở bàn lớn.
- Heuristic có ích khi không gian tìm kiếm lớn; ở bài dễ, chi phí tính heuristic có thể lấn phần tiết kiệm được.

Hướng tiếp theo: thêm bàn khó và vô nghiệm, đo nhiều seed hơn, đo riêng tác dụng từng kỹ thuật.

Tài liệu tham khảo

1. Luật Pipes: `https://www.puzzle-pipes.com/`.
2. Luật Akari: `https://www.nikoli.co.jp/en/puzzles/akari/`.
3. Russell và Norvig, *Artificial Intelligence: A Modern Approach*, chương 3 và 6.
4. Bram Pulles (2021), *Analysis of Akari*, mục 4.1.1 và 4.1.3.
5. Victor Dekker (2015), *A Difficulty Measure for Light Up Puzzles*.

Danh sách đầy đủ trong báo cáo.